BFI LOAN ORIGINATION · BRAVO WORKFLOW ARCHITECTURE · EFFORT BRIEF FOR THE CTO

What best-practice workflow costs

THE ANSWER, IN ONE LINE

≈9–14 engineer-months to take every product to the target shape — but the first 2–3 engineer-months buy almost all of the governance value and move no workflow at all.

Rp265M–700M for the full programme; **Rp60M–160M for the part that is unconditional.** Against 30–57 engineer-months for either platform migration.

Question asked What is the effort to move Bravo's workflows to best practice? **Evidence** `ms-bpm` working tree · production BPM Postgres, 90d to 2026-09-09 **As at** 2026-09-11

The premise most people hold about Bravo's workflows is false in production

"A single giant workflow handles all products" describes a spine that carries **5.5% of applications and exactly one product.** The complexity being paid today is the *legacy per-product* design — which is also, almost exactly, the design usually proposed as the fix.

ROOT WORKFLOW	STARTED, 90D	SHARE	GENERATION
`NDF2W`	**248,682**	73.4%	legacy monolith — 174 nodes, 359 commits
`NDF4W`	**59,909**	17.7%	legacy monolith — 223 nodes, 392 commits
`NDF4W_RO`	11,458	3.4%	legacy variant
`Unified_Process_Main_Workflow`	18,809	**5.5%**	**the shared spine** — 43 nodes, 0 user tasks

Of the 18,806 applications on the shared spine, **every one is DF4W** — plus a 4-application NDF4W company pilot. DF2W is fully configured with zero volume because it has not been released: it is in UAT with an LOS penetration test running.

THE GIANT WORKFLOWS ARE THE PER-PRODUCT ONES

The unified spine is **43 top-level nodes over 33 files**. `NDF4W` is **223** and `NDF2W` is **174**. The legacy generation *is* the per-product design, and it is where the complexity accumulated: **156 distinct delegate beans** across the legacy roots against **69** for the unified generation's five products.

THERE IS ALREADY A WORKING STRANGLER BEACHHEAD

The spine has run DF4W in production **for months, flat, at 5.5%**. Nobody planned it as a migration — but it proves the target shape carries real volume. The programme below is largely about **making that deliberate.**

Product variation is expressed in four places, and the model shows none of them

This — not the size of any diagram — is what the effort below is buying out. **No artifact in the repository shows what a given product actually executes.**

MECHANISM	WHERE	MEASURED TODAY	WHY IT IS A GAP
Gateway conditions on product	`unified-main-workflow`, `-underwriting`, `-pd-model`	**3 files** carry `applicationWorkflowSelectorType`; **9 distinct feature flags** read from BPMN via `environment.getProperty`	Product routing is mixed with feature flags **inside process XML**; the diagram reads as one flow but executes as several
Config-gated no-op tasks	`BaseActivity.isNeedToProceed` + the `workflow_*` tables	**55 classes gated**; **64 workflow-config migrations** since 2024-03	A skipped task **completes normally**, so Camunda history, Cockpit and the BPMN all show steps that did nothing. The only way to know a product's real path is a **five-table join**
Product branches in Java	`activity/unified/**`	**19 of 68** classes; **42 call sites** — `isDF4W()` 21, `isDF2W()` 7, `isNDF2W()` 7, `isNDF4W()` 5, `isDF2WSharia()` 2	The same activity behaves differently per product **with no trace in BPMN or config**. This is the layer that grows silently
Feature flags per product	`application.yaml`	`featDF2W`, `featDF2WSharia`, four `featBypassScoringProcess*` — inside **726** `feat*` **keys** in total	A product can be **half-enabled per environment**, and the BPMN cannot express that

THE PHANTOM TASKS ARE MEASURED, NOT HYPOTHETICAL

In `Unified_Process_KYC_Check`, Address Verification, Phone Checking and Shopee Score each ran **4,822 times in 7 days with 86–89% of executions under 50 ms** — below the network floor. They are active for DF4W only; for every other product they log "Skipping" and return, while history records a completed step. The real calls in the same process sit at **898–1,874 ms**.

Eight principles define best practice. Bravo meets one and a half.

PRINCIPLE	CURRENT STATE	SEVERITY
A Model is the source of truth; structure in the model, data in config	Structure encoded as `is_active` no-op rows; 55 classes gated; effective path **not derivable from the model**	**High**
B A thin product-owned spine per product	One shared spine for all five; product chosen by gateways and a selector variable. **No product owns a readable spine**	**High**
C Domain children shared, with no product logic inside	19 of 68 unified activities branch on `application.isXxx()` — 42 call sites	**High**
D Fork a child only on structural difference	**Done once correctly** — `…Underwriting_Regular`. Everywhere else variation is config no-ops. **The good precedent is the exception**	Medium
E Product kept out of domain code behind stable contracts	Config + Java + gateways + flags all carry product: **four mechanisms to change to alter one product's flow**	**High**
F Least-privilege engine, per-product deployment unit	`/camunda` is `permitAll`; `/engine-rest` grants full engine rights to any holder of the shared key — **the actual RCE vector**. One deployment unit for all products	**High** (security)
G Migrate by strangler, never big-bang	**Happening** — but **by accident**, flat at 5.5%, with no migration plan. Legacy carries ~94% and the whole retail book	Medium
H Dead and duplicated assets removed	Four dead `setting.workflow.map` keys; one unreferenced mock deployment; **10 named 2W/4W delegate pairs** doing the same job twice	Low–Med

> **Bravo has the right building blocks and one correct fork — it just expresses product variation in three places the model cannot show.**
>
> So no product has a readable, owned, independently-deployable spine, and the legacy monoliths carrying 94% of volume were never migrated at all.

Product-owned spines, domain-owned children, explicit variation

KEEP — WHAT THE UNIFIED DESIGN GOT RIGHT

— **Orchestrators contain no user tasks.** Human work is confined to leaf children — confirmed in production: the orchestrators recorded **zero user-task instances** in 90 days

— **Domain steps exist once.** Onboarding DF2W Sharia in 2026 touched the spine in one commit plus migrations, instead of adding a fourth monolith

— **Escalation, cancellation and rejection are centralised** as boundary events on the spine, not re-drawn per product

— **The correct fork already exists:** `Unified_Process_Workflow_Underwriting_Regular` is a DF2W-family fork called from the shared underwriting orchestrator. The pattern is known — it is simply not applied consistently

CHANGE — THE FIVE RULES, IN ORDER OF PAYOFF

— **1 · One thin spine per product** (~40 nodes): the source of truth for stage order and which child each stage calls. The spine *is* the selector; retire the gateways

— **2 · No silent skipping.** A task not applicable to a product must not be on that product's path. At minimum, a missing config row must **fail loudly**

— **3 · Remove `application.isXxx()` from the 19 shared activities.** Product behaviour belongs in the spine or a product-specific child, never in a shared bean

— **4 · Fork a domain child only on structural difference** — order of steps, different human roles, different partners. A different threshold does not qualify

— **5 · Publish the effective flow per product**, generated on every build and diffed in CI

CAMUNDA 7 MAKES THE MIDDLE PATH CHEAP

A call activity's `calledElement` can be **an expression** — so a product spine dispatches to the shared `survey` child for most products and to `survey_sharia` for one, **with no gateway and no flag**. Five spines of ~40 nodes cost far less than the 25 per-product children a full split would require — and full duplication is exactly how `NDF4W` and `NDF2W` drifted apart.

How these numbers were built — and how much to trust each phase

BASIS

Engineer-**days** of a competent engineer **already familiar with** `ms-bpm`, including that engineer's own testing, **excluding UAT run by business users**. 21 working days to the engineer-month; Rupiah at the pack's standing assumption of **Rp30–50M fully loaded per engineer-month** — substitute BFI's rate card. Ranges are **low-to-high, not confidence intervals**.

Unlike the end-of-support numbers, these are not a team memo. They are derived here from counted inventory — delegates, branch sites, gated classes, user tasks, production volumes — and should be re-estimated by the squads that would do the work.

THE ONE MEASUREMENT THAT WOULD FIRM UP PHASES 3–4

Of `NDF2W`'s **67 delegate beans, exactly one** (`checkUnderwritingNeededUnifiedActivity`) is also used by the unified children. **So migration is not re-pointing: it is establishing, bean by bean, whether the unified child already does that job.**

A delegate-level mapping — **5–8 days** — would convert the widest range in this deck into a real number. It is the direct analogue of the LORA coverage-gap inventory.

CONFIDENCE, BY PHASE

PHASE	CONFIDENCE	WHY
● P1 VISIBILITY	**High**	The queries exist (§8.4) and the parse scripts exist. Tooling work with a known shape
● P2 PROVE ON DF4W	**Medium-high**	Scope is enumerable: one spine, 3 gateways, 21 `isDF4W()` sites, a known set of gated KYC/RAC steps
● P3 NDF2W	**Low-medium**	Turns on how much of NDF2W's behaviour the shared children already cover — **unmeasured**. The widest range here
● P4 THE REST	**Low**	Scaled from P3 plus counted inventory; Sharia is a separate deployment nobody has scoped

WHAT IS EXCLUDED FROM EVERY NUMBER

Business-run UAT, product-owner workshop time, domain-logic rewrites (survey and underwriting *behaviour* is out of scope), and any change to the LORA or Temporal track.

See it and stop the bleeding — no workflow moves

WORK ITEM	WHAT IT INVOLVES	ENG-DAYS	EXIT CRITERION
Publish the effective per-product flow	Join `workflow_master_config_detail` ⋈ `selector_order` ⋈ `selector_type` and overlay on the BPMN; one diagram per product, regenerated on build. **The §8.4 query and the parse scripts already exist**	**8–13**	Every live product has a current, owner-readable flow diagram
Freeze new hidden variation	PR check failing a new `application.isXxx()` in `activity/unified/**`, and a new config-gated class with no doc entry	**2–4**	CI blocks both; the count of branch sites can only go down from 42
Harden the engine	Authenticate and network-restrict `/camunda` **and** `/engine-rest`; rotate `INTERNAL_SERVICE_KEY`; register a `ProcessEngineAuthenticationFilter`; purge the 61 injected definitions	**3–5** already in Decision A	Injected definitions purged, verified in prod and Sharia
Decide the target	A one-page ADR — product-owned spines, shared domain children, fork-on-structure — with product-owner sign-off	**2–3**	ADR merged
Cheap cleanup	Delete the four dead `setting.workflow.map` keys; remove `Process_NDF4W_Scoring_1_Mock_Ro`	**0.5–1**	Map keys map 1:1 to real processes
Phase 1 total — 1–2 engineers, ~1 month elapsed		**15–26 days ≈0.8–1.3 eng-months**	**Rp 25M – 65M**

WHAT PHASE 1 DELIBERATELY DOES NOT DO

No workflow moves, no BPMN is restructured, and no product migrates. Phase 1 changes **what can be seen and what can be added** — it makes each product's real path visible and stops the count of hidden branches from rising. Every structural change is deferred to Phase 2, behind a signed-off ADR.

THIS PHASE IS UNCONDITIONAL — IT SURVIVES EVERY DECISION B OUTCOME

The per-product effective-flow artifact is **exactly the input the LORA gap inventory needs** ("every Bravo delegate, status, assignment rule and approval path mapped to a LORA ProcessStep"). If BFI migrates to LORA, this work is **a prerequisite, not a write-off**. If it ports to Temporal, a faithful port of an *unreadable* model ports the problem — this makes the port smaller. And if Bravo stays, it is the foundation of everything below.

Prove the pattern on DF4W — the only product already on the spine

Lowest-risk possible pilot: **5.5% of volume, one product, already in production on the shared spine.** If the target shape does not work here, it does not work.

WORK ITEM	COUNTED SCOPE	ENG-DAYS	EXIT CRITERION
Build `spine_df4w`	A thin executable spine naming DF4W's stages and calling each domain child by `calledElement`; new DF4W volume routed behind a flag	**5–8**	New DF4W loans run `spine_df4w`; the shared spine no longer receives DF4W
Remove DF4W's hidden variation	**21 `isDF4W()` call sites** in shared beans, plus the **3 `applicationWorkflowSelectorType` gateways** on the DF4W path	**6–10**	Zero product gateways and zero `isDF4W()` on the DF4W path
Kill the phantom tasks	Convert the gated KYC and RAC no-ops into explicit model choices — a gateway where the choice is data-driven, a fork where it is product-driven	**8–12**	No sub-50 ms no-op service tasks in DF4W history for the gated KYC steps
Lock in observability	Per-product flow diagram generated and **diffed in CI**; contract tests per domain child	**4–6**	A change to a shared child that alters DF4W's path appears as a diagram diff in review
Route, soak and prove parity	Strangler routing, rollback path, latency and error parity against the shared spine	**4–6**	Parity held over a full soak window at DF4W's ~6,800 applications/month
Phase 2 total — 1–2 engineers, ~1–1.5 months elapsed		**27–42 days ≈1.3–2.0 eng-months**	**Rp 40M – 100M**

WHAT PHASE 2 SETTLES THAT NO DOCUMENT CAN

Whether one thin spine plus shared children really carries a product without the config gating — and **what the true cost per product is**. Phases 3 and 4 should be re-estimated against Phase 2's actuals rather than the ranges on the next two slides.

NDF2W is the prize and the expense — 248,682 applications a quarter

WORK ITEM	ENG-DAYS	COUNTED SCOPE
Map NDF2W's delegates onto shared children; build `spine_ndf2w`	**10–15**	67 delegate beans, 174 top-level nodes, 45 embedded subprocesses
Extend the domain children to cover NDF2W semantics	**25–40**	**66 of the 67 beans have no unified counterpart.** The single widest range in the programme
Re-home the human work	**8–12**	20 user tasks — including the 185,574-instance User Survey Task — whose assignment lives in Java, since `candidateGroups` is empty on every task
Strangler routing, dual-run, parity harness, runbook	**10–15**	Legacy `NDF2W` stays live for in-flight instances and rollback
Regression and UAT support	**8–12**	Engineer-side only; business UAT excluded
Phase 3 total — 2 engineers, ~3–4 months elapsed	**61–94 days ≈3.0–4.5 eng-months**	**Rp 90M – 225M**

1 of 69

UNIFIED DELEGATE BEANS ALSO USED BY A LEGACY ROOT — THE REASON MIGRATION IS NOT RE-POINTING

≈25%

EXIT GATE: SHARE OF NEW NDF2W STARTS ON THE NEW SPINE, AT ERROR AND LATENCY PARITY

WHY THIS NUMBER IS HONEST BUT WIDE

The two generations share **almost no implementation**. Every NDF2W step has to be classified: *the shared child already does this*, *the shared child needs extending*, or *NDF2W genuinely differs and needs a fork*. Nobody has done that classification — it is the 5–8 day mapping on slide 6, and it is worth doing **before** committing to this phase.

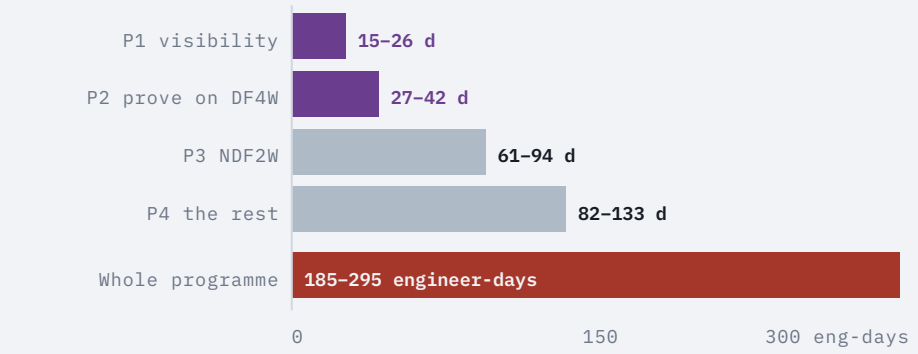
THE PAYOFF IS PROPORTIONAL

NDF2W alone is **73% of Bravo's originations**. Moving it is what converts the programme from a governance improvement into a reduction in the maintenance surface that 359 commits on one file represent.

The rest of the estate, and what the whole thing costs

PHASE 4 WORK ITEM	ENG-DAYS	SCOPE
`NDF4W` + `NDF4W_RO` onto spines	50–80	68 + 14 delegate beans, 223 top-level nodes, 21 user tasks; RO reuses NDF4W's children (17 of 19 steps identical)
Sharia's separate deployment	15–25	Its own engine and deployment unit; 12 delegates, never scoped
De-duplicate the 2W/4W beans	12–20	**10 named pairs** — `createCif` / `createCif2w`, `preFatalRac` / `…2w`, `addressVerification` / `…2W` and seven more — behind shared implementations
Launch DF2W on `spine_df2w`	5–8	Incremental only — it is configured and in UAT, not dead config
Phase 4 total — 2 engineers, ~4–6 months elapsed	**82–133 days ≈4.0–6.3 eng-months**	**Rp 120M – 315M**

PROGRAMME, CUMULATIVE



STAFFING ASSUMPTION BEHIND THE ELAPSED FIGURES

Phases 1–2 assume **1–2 engineers**, Phases 3–4 **2 engineers** running alongside normal delivery. At that shape the programme spans roughly **12–18 months elapsed**; with three engineers on Phases 3–4 it compresses toward 9–12 without changing the engineer-day totals.

PROGRAMME TOTAL

185–295 engineer-days ≈ 9–14 engineer-months ≈ Rp265M–700M

Against 30–57 **engineer-months** for either platform migration, and 18–33 **engineer-days** for the end-of-support fix that must happen first.

Where this sits against the other things competing for the same engineers

PROGRAMME	EFFORT	INDICATIVE RP	WHAT IT BUYS	CONDITIONAL ON?
Decision A — end-of-support fix (fork + Spring Boot 4 + RCE)	**18–33 eng-days**	**Rp45–125M**	A supported engine and a supported Spring Boot; closes a live code-execution path	**Nothing** do it first
● WORKFLOW P1 ● P2 — visibility, freeze, prove on DF4W	**42–68 eng-days** ≈2.0–3.2 eng-months	**Rp60–160M**	An owner-readable flow per product, CI that stops the hiding, and the target shape proven on live volume	**Nothing** useful under every option
● WORKFLOW P3 ● P4 — migrate the monoliths	**143–227 eng-days** ≈6.8–10.8 eng-months	**Rp205–540M**	Every product on its own readable, independently-deployable spine; the duplication retired	**Bravo's horizon** and Decision B
Option 2 — migrate Bravo to Temporal	31–57 eng-months	Rp930M–2.85B	Removes the workflow-engine vendor permanently; **preserves the architecture exactly**	Decision B
Option 3 — migrate Bravo to LORA	30–57 eng-months	Rp900M–2.85B	One platform; adds an automated step with no orchestration edit	Decision B

> **The whole workflow programme costs about a quarter of either migration — and it is the only one of the three that improves Bravo's design without changing platform.**
>
> It is also the only one whose **first half is worth funding even if BFI eventually leaves Bravo**: a per-product effective-flow map is the prerequisite for the LORA gap inventory, and a legible model is what makes a Temporal port smaller rather than a faithful port of the same confusion.

option-summary.md §3, §7 · workflow-gap.md §12 · effort derived 2026-09-11

What the status quo costs, measured rather than asserted

BLAST RADIUS ON EVERY CHANGE

Commit subjects on `unified-main-workflow.bpmn` name **DF4W, DF2W, Sharia, RO, NDF4W and Company**. Every one of those redeployed the spine that **all five products** run. On the survey child, DF4W work changed the orchestration NDF2W also uses. One deployment unit means one shared risk on every product edit.

A TEST MATRIX NOBODY CAN ENUMERATE

Correctness of one BPMN file depends on **products × config rows × flags**. The **64 workflow-config migrations** include repeated corrective `set-active` / `set-inactive` pairs — `pd-model-df4w-set-active`, `underwriting-return-set-inactive-except-df4w`. That is what a hidden matrix looks like in practice: **~2 corrective migrations a month** since 2024-03.

SILENT DIVERGENCE, NOT LOUD FAILURE

With `BaseActivity` returning quietly, **a missing config row does not fail a deployment or a process. It removes a check from a loan product** — and history still shows the step completing. There is no alert for this class of defect today.

NO ARTIFACT A PRODUCT OWNER CAN BE HANDED

A stakeholder cannot be shown a diagram of their product: the diagram is shared and the differences live in SQL and Java. **Business ownership per product is not modelled anywhere** — `candidateGroups` is empty on every user task and assignment happens in code.

DUPLICATION KEEPS COMPOUNDING

The legacy generation holds **156 distinct delegate beans** and the two big monoliths share only **20** — most of the rest is the same domain step under a suffixed name. **Full per-product duplication is how NDF4W and NDF2W drifted apart**, and the pattern continues wherever a product needs one different step.

WHAT IS *NOT* A REASON TO ACT

Engine cost. Bravo's orchestration tier is the **cheapest of the three platform options at ≈Rp58M/month**. This programme is about legibility, blast radius and correctness — **not savings**. It will not pay for itself in infrastructure.

What could make these numbers wrong, and what must be true before starting

RISK	EFFECT ON THE ESTIMATE	MITIGATION
Shared-child coverage for NDF2W is worse than assumed	Phase 3 is the widest range for exactly this reason; a bad case pushes it past 94 days	Run the **5–8 day delegate mapping first**; re-estimate P3/P4 against Phase 2 actuals
No engine-layer safety net	Every phase is riskier than its day count implies	**4 of `ms-bpm`'s 1,457 test files run a Camunda process.** The parity harness in Decision A is a prerequisite here too, not a nice-to-have
In-flight instances during cutover	Adds calendar, not effort, if planned; adds both if not	Strangler by construction — legacy roots stay deployed for in-flight work and rollback
Decision B lands on LORA mid-programme	Phases 3–4 become sunk	**Gate P3 on a published Bravo horizon.** P1–P2 remain useful either way
Product owners cannot be assembled to sign off the target	Blocks the ADR, which gates P2	Two squads and three consoles already own these products — the seam exists; use it

ONE HARD PREREQUISITE

The engine hardening is **not optional and not sequenced late.** `/camunda` is `permitAll` and `/engine-rest` grants full engine rights to any holder of the shared key — the vector by which **61 hostile process definitions** reached production. **Nothing else on this plan should start while a third party can deploy process definitions into the engine being restructured.**

EXPLICITLY OUT OF SCOPE

Rewriting survey or underwriting **domain logic**; migrating Sharia's separate deployment beyond the spine work costed in Phase 4; and any change to the LORA or Temporal track. This plan is only about **the shape of Bravo's workflows** and moving product variation out of hiding.

Fund the first half now; gate the second half on Bravo's horizon

1	**Approve Phases 1 and 2 as an unconditional spend.** Visibility, a CI freeze on new hidden variation, and the target shape proven on DF4W — the product already running on the spine. No legacy workflow moves, and the output is a prerequisite under every Decision B outcome.	42–68 ENG-DAYS ≈2–3 ENG-MONTHS RP60–160M
2	**Commission the 5–8 day delegate mapping before deciding on Phase 3.** It converts the widest range in this deck — how much of NDF2W the shared children already cover — into a real number, and it is the direct analogue of the LORA coverage-gap inventory.	5–8 DAYS DO IT INSIDE P1
3	**Hold Phases 3 and 4 against the published Bravo horizon.** Migrating 94% of volume onto product spines is worth 143–227 engineer-days only if Bravo has years left. That is the same input Decision B needs, and it is a business call.	143–227 ENG-DAYS RP205–540M GATED
4	**Accept the monthly metrics as the reporting contract** — and re-baseline Phases 3–4 against Phase 2's actuals rather than the ranges in this deck.	MONTHLY

PROGRAMME METRIC, REPORTED MONTHLY	TODAY	TARGET
% of new-application volume on explicit product spines	**0** — DF4W-on-shared-spine does not count	all live products
`application.isXxx()` call sites in `activity/unified/**`	**42** across 19 of 68 classes	**0**
Config-gated no-op service-task executions per day	heavy — 89% of KYC gated steps under 50 ms	near-zero
Distinct delegate beans across the legacy products	**156**	falling
Products with a current owner-readable spine diagram	**0**	all live products

Sources: workflow-gap.md (the gap, verified against the production BPM database) · workflow-analysis.md (what exists) · option-summary.md (the platform options and cost basis) · effort derived 2026-09-11 · bravo-analysis/docs